
TinkerWithMe: Arduino Robotics & Electronics

Lesson Plan — Automated Fan Control (Core Project) with Simon Says (Extension/Next Session)

Age Group: 13–15 years | **Duration:** 90 minutes | **Level:** Advanced

*Note to facilitators: The Simon Says game (120 min) cannot be completed to a working gameplay state within this 90-minute slot alongside the Fan Control build. This plan makes **Fan Control** the primary, fully-completed build (matches the 90-min session length) and positions **Simon Says** as a guided starter activity for fast finishers, to be completed in the next session. Full instructions, code, and assessment criteria are provided for both projects.*

1. Session Overview

Students will design, wire, and program an **automatic temperature-controlled fan system** — the same principle used in real-world cooling systems for computers, greenhouses, and electronics enclosures. They will read live temperature data from a sensor, make decisions in code based on that data, and use a relay to safely switch a fan on and off. This project reinforces the core engineering loop: *sense* → *decide* → *act*, a foundational concept in robotics and automation.

Students who finish early will begin prototyping a second advanced project — the **Simon Says memory game** — building their wiring and testing individual LED/button/buzzer components in preparation for a full build next session.

2. Learning Objectives

By the end of this session, students will be able to:

- **Recall** the function of a relay module and explain why it is used to control higher-power devices from a microcontroller (Knowledge).
- **Explain** how a temperature sensor converts a physical property into digital data an Arduino can process (Comprehension).
- **Construct** a complete sensor-to-actuator circuit on a breadboard using a DHT sensor, relay module, and fan (Application).
- **Write and modify** Arduino code that uses conditional logic (if/else) and a threshold variable to automate a real-world action (Application/Analysis).
- **Debug** a non-functioning circuit or sketch systematically using serial monitor output and a troubleshooting checklist (Analysis/Evaluation).

3. Materials & Setup

Quantities below assume groups of 2–3 students. Adjust multiples for total group size.

Per Group — Fan Control (Core Project)

ITEM	SPECIFICATION	QTY
Microcontroller board	Arduino Uno R3 (or compatible)	1
USB cable	USB-A to USB-B, 1m	1
Breadboard	Half-size, 400 tie points	1
Temperature/humidity sensor	DHT11 (or DHT22) digital sensor module	1
Relay module	5V single-channel relay module (opto-isolated)	1
DC fan	5V–12V small DC brushless fan with separate power leads	1
External power source for fan	4xAA battery pack (6V) or 9V battery with barrel/leads, matching fan voltage	1
Indicator LED	5mm, red	1
Resistor	220Ω, ¼W	1
Jumper wires	Male-to-male, assorted lengths	15

Jumper wires	Male-to-female	4
--------------	----------------	---

Per Group — Simon Says (Extension / Next Session)

ITEM	SPECIFICATION	QTY
LEDs	5mm, colours: red, green, blue, yellow	4
Resistors	220Ω, ¼W	4
Push buttons	12mm momentary tactile switch	4
Resistors (pull-down)	10kΩ, ¼W	4
Piezo buzzer	5V passive/active buzzer	1
Breadboard	Full-size, 830 tie points	1
Jumper wires	Male-to-male, assorted	20

Shared / Classroom Resources

- Laptop with Arduino IDE pre-installed per group (1 per group)
- Printed circuit diagrams (1 per group, both projects)
- Whiteboard or projector for live code walkthrough
- Multimeter (1–2 shared for troubleshooting)
- Spare Arduino Unos, LEDs, and resistors (for quick swaps)
- Safety note cards on relay/mains awareness (see Classroom Management)

Pre-Session Setup Checklist

- Install/verify Arduino IDE and DHT sensor library (DHT.h and Adafruit Unified Sensor) on all laptops
- Pre-test one full Fan Control circuit as a reference/demo model
- Print wiring diagrams and code handouts
- Charge/check battery packs for fan power
- Arrange desks in pods of 2–3 for collaborative building

4. Minute-by-Minute Schedule

TIME	DURATION	ACTIVITY
------	----------	----------

0:00– 0:05	5 min	Hook & Welcome: Ask "What in your house automatically turns on based on heat or light?" Discuss fridges, AC, streetlights. Introduce today's goal: build a smart fan.
0:05– 0:15	10 min	Concept Introduction: Explain sensors, relays, and the sense-decide-act loop. Show relay module up close; explain isolation between Arduino and fan power for safety.
0:15– 0:22	7 min	Circuit Diagram Walkthrough: Distribute diagrams. Facilitator demonstrates wiring on the reference model, pointing out DHT and relay pinouts.
0:22– 0:50	28 min	Build Phase 1: Groups wire DHT sensor and indicator LED first; facilitator circulates to check connections before power-on.
0:50– 0:55	5 min	Stretch Break: Quick standing break; re-energize before relay wiring (relay wiring done fresh, not rushed).
0:55– 1:05	10 min	Build Phase 2: Wire relay module and fan circuit. Facilitator double-checks relay wiring on every group before power is connected.
1:05– 1:15	10 min	Code Upload & Calibration: Upload provided sketch, open Serial Monitor, observe live temperature readings, adjust threshold value together.
1:15– 1:22	7 min	Test & Debug: Groups test fan trigger by warming sensor (breath/hands). Facilitator supports debugging using checklist.
1:22– 1:27	5 min	Showcase: Each group demonstrates their working fan trigger to the class (30 seconds each).
1:27– 1:30	3 min	Wrap-Up: Recap objectives, preview Simon Says for next session, clean-up and component count.

Early finishers who complete Fan Control before 1:22 move directly into the Simon Says starter activity (see Extensions).

5. Project Breakdowns

5.1 Fan Control (Core Project — Advanced, 90 min)

Step-by-Step Instructions

1. Place the Arduino and breadboard side by side. Insert the DHT sensor into the breadboard with its 3 pins spanning separate rows.
2. Connect DHT VCC to Arduino 5V, GND to GND, and the Data pin to Arduino digital pin 2 (through a 10kΩ pull-up resistor to 5V if your module doesn't have one built in).
3. Wire the indicator LED: anode through a 220Ω resistor to digital pin 3, cathode to GND.

4. Wire the relay module: VCC to Arduino 5V, GND to Arduino GND, IN pin to digital pin 7.
5. On the relay's high-power side (screw terminals), connect the fan's positive lead through the relay's **NO (Normally Open)** and **COM** terminals. Connect the external battery pack positive to COM, fan positive to NO, and fan negative directly to battery negative.
6. **Do NOT connect the fan circuit to mains electricity.** Use only low-voltage DC battery power for this build.
7. Open Arduino IDE, install "DHT sensor library" and "Adafruit Unified Sensor" via Library Manager if not already installed.
8. Upload the provided sketch below. Open Serial Monitor (9600 baud) to view live temperature readings.
9. Adjust the `tempThreshold` variable in code to a value slightly above current room temperature, then test by warming the sensor with cupped hands.
10. Confirm the relay clicks and the fan spins when threshold is exceeded, and stops when temperature drops again.

Full Working Code

```
/*
  TinkerWithMe - Automated Fan Control
  Reads temperature from a DHT sensor and switches a fan
  on/off via a relay module based on a threshold value.

  Wiring:
  DHT Data  -> Pin 2
  LED       -> Pin 3 (through 220 ohm resistor)
  Relay IN  -> Pin 7
*/

#include "DHT.h"           // Include the DHT sensor library

#define DHTPIN 2           // DHT sensor data pin
#define DHTTYPE DHT11      // Change to DHT22 if using that sensor
#define LEDPIN 3           // Indicator LED pin
#define RELAYPIN 7         // Relay control pin

DHT dht(DHTPIN, DHTTYPE); // Create DHT sensor object

float tempThreshold = 28.0; // Fan turns ON above this temperature (Celsius)
                          // Adjust this value based on room conditions

void setup() {
  Serial.begin(9600);       // Start serial communication for debugging
  dht.begin();              // Initialize the DHT sensor
  pinMode(LEDPIN, OUTPUT);  // Set LED pin as output
  pinMode(RELAYPIN, OUTPUT); // Set relay pin as output
```

```

digitalWrite(RELAYPIN, LOW); // Ensure fan starts OFF
}

void loop() {
  float temperature = dht.readTemperature(); // Read temp in Celsius

  // Check if the reading failed (common with loose wiring)
  if (isnan(temperature)) {
    Serial.println("Failed to read from DHT sensor! Check wiring.");
    delay(2000);
    return; // Skip the rest of this loop cycle
  }

  // Print the current temperature to Serial Monitor
  Serial.print("Temperature: ");
  Serial.print(temperature);
  Serial.println(" C");

  // Decision logic: sense -> decide -> act
  if (temperature > tempThreshold) {
    digitalWrite(RELAYPIN, HIGH); // Turn fan ON
    digitalWrite(LEDPIN, HIGH); // Turn indicator LED ON
    Serial.println("Fan: ON");
  } else {
    digitalWrite(RELAYPIN, LOW); // Turn fan OFF
    digitalWrite(LEDPIN, LOW); // Turn indicator LED OFF
    Serial.println("Fan: OFF");
  }

  delay(2000); // Wait 2 seconds before next reading (DHT11 needs this delay)
}

```

Common Mistakes & Fixes

- **Serial Monitor shows "nan":** DHT data pin loose or missing pull-up resistor. Re-seat wires and check resistor placement.
- **Relay clicks but fan doesn't spin:** Battery pack not connected or wired to wrong relay terminal (should use NO + COM, not NC).
- **Fan spins constantly:** Threshold set too low, or relay wired to Normally Closed (NC) instead of Normally Open (NO) terminal.
- **Arduino resets when fan turns on:** Fan drawing power through Arduino instead of separate battery — check external power wiring.
- **Library errors on compile:** DHT library not installed — check Library Manager under Sketch → Include Library.

Success Criteria

- Circuit is wired correctly with no loose connections and Arduino IDE compiles/uploads without errors.
- Serial Monitor displays continuously updating, accurate temperature readings.

- Relay and fan activate reliably when temperature crosses the threshold, and turn off when it drops.
- Student can explain, in their own words, the sense-decide-act logic used in their code.

5.2 Simon Says Memory Game (Extension / Next Session — Advanced, 120 min)

In this 90-minute session, early finishers will only complete Steps 1–3 (wiring and component testing). Full gameplay logic and code will be completed in the next session.

Step-by-Step Instructions (Session 1 Portion)

1. Insert all 4 LEDs into the breadboard, each with its own 220Ω resistor to a separate digital pin (e.g., pins 2, 3, 4, 5).
2. Wire all 4 push buttons with 10kΩ pull-down resistors to separate digital pins (e.g., pins 8, 9, 10, 11).
3. Wire the piezo buzzer's positive lead to pin 12 and negative to GND.
4. Upload the test sketch below to confirm every LED lights and every button/buzzer responds individually before building game logic.
5. (*Next session*) Add the full game logic: random sequence generation, playback, and player input checking.

Full Working Code (Complete Game — for next session's build)

```
/*
  TinkerWithMe - Simon Says Memory Game
  Classic memory game: Arduino plays a growing sequence of
  LEDs+tones, player must repeat it correctly to advance levels.

  Wiring:
  LEDs    -> Pins 2, 3, 4, 5 (through 220 ohm resistors)
  Buttons -> Pins 8, 9, 10, 11 (with 10k pull-down resistors)
  Buzzer  -> Pin 12
*/

const int ledPins[4]    = {2, 3, 4, 5};      // LED output pins
const int buttonPins[4] = {8, 9, 10, 11};    // Button input pins
const int buzzerPin     = 12;                // Buzzer pin

const int tones[4] = {262, 294, 330, 349};  // Musical notes per LED (C,D,E,F)

int sequence[100];      // Stores the growing game sequence
int level = 0;          // Current level (sequence length)

void setup() {
  Serial.begin(9600);
```

```

    for (int i = 0; i < 4; i++) {
        pinMode(ledPins[i], OUTPUT);
        pinMode(buttonPins[i], INPUT);
    }
    pinMode(buzzerPin, OUTPUT);
    randomSeed(analogRead(A0)); // Seed randomness using floating analog pin
    startNewGame();
}

void loop() {
    playSequence(); // Show the current sequence to the player
    if (!getPlayerInput()) { // If player makes a mistake...
        gameOver();
        startNewGame();
    } else {
        level++; // Correct! Advance to next level
        delay(800);
    }
}

// Adds one new random step and starts the game fresh
void startNewGame() {
    level = 1;
    for (int i = 0; i < 100; i++) {
        sequence[i] = random(0, 4); // Fill sequence with random LED indices (0-3)
    }
    Serial.println("New game started!");
}

// Lights up and sounds each step in the current sequence
void playSequence() {
    for (int i = 0; i < level; i++) {
        int idx = sequence[i];
        digitalWrite(ledPins[idx], HIGH);
        tone(buzzerPin, tones[idx], 300);
        delay(400);
        digitalWrite(ledPins[idx], LOW);
        delay(200);
    }
}

// Waits for player to press buttons matching the sequence
// Returns true if correct, false if a mistake is made
bool getPlayerInput() {
    for (int i = 0; i < level; i++) {
        int pressed = waitForButtonPress();
        digitalWrite(ledPins[pressed], HIGH);
        tone(buzzerPin, tones[pressed], 200);
        delay(200);
        digitalWrite(ledPins[pressed], LOW);

        if (pressed != sequence[i]) {
            return false; // Wrong button - game over
        }
    }
    return true; // All correct!
}

// Waits until one of the 4 buttons is pressed, returns its index
int waitForButtonPress() {

```



```

while (true) {
  for (int i = 0; i < 4; i++) {
    if (digitalRead(buttonPins[i]) == HIGH) {
      delay(50); // Simple debounce
      while (digitalRead(buttonPins[i]) == HIGH); // Wait for release
      return i;
    }
  }
}

// Plays a "fail" tone and flashes all LEDs
void gameOver() {
  Serial.print("Game Over! You reached level: ");
  Serial.println(level);
  for (int i = 0; i < 3; i++) {
    for (int j = 0; j < 4; j++) digitalWrite(ledPins[j], HIGH);
    tone(buzzerPin, 100, 300);
    delay(300);
    for (int j = 0; j < 4; j++) digitalWrite(ledPins[j], LOW);
    delay(200);
  }
  delay(1000);
}

```

Component Test Sketch (For This Session's Wiring Check)

```

// Quick test: confirms each LED, button, and the buzzer work individually
const int ledPins[4]    = {2, 3, 4, 5};
const int buttonPins[4] = {8, 9, 10, 11};
const int buzzerPin     = 12;

void setup() {
  for (int i = 0; i < 4; i++) {
    pinMode(ledPins[i], OUTPUT);
    pinMode(buttonPins[i], INPUT);
  }
  pinMode(buzzerPin, OUTPUT);
}

void loop() {
  for (int i = 0; i < 4; i++) {
    if (digitalRead(buttonPins[i]) == HIGH) {
      digitalWrite(ledPins[i], HIGH);
      tone(buzzerPin, 300, 100);
    } else {
      digitalWrite(ledPins[i], LOW);
    }
  }
}

```

Common Mistakes & Fixes

- **Button always reads HIGH:** Missing pull-down resistor — check 10kΩ resistor is wired from button pin to GND.

- **LED doesn't light:** Reversed polarity — flip LED so longer leg (anode) faces the resistor/pin side.
- **Buzzer silent:** Using `digitalWrite` instead of `tone()`, or wrong pin defined in code.
- **Multiple buttons trigger at once:** Loose wiring causing crosstalk — check breadboard rows aren't shared unintentionally.

Success Criteria (Session 1 Portion)

- All 4 LEDs light individually when tested.
- All 4 buttons register presses correctly in Serial Monitor or via test sketch.
- Buzzer produces distinct tones per test.
- Circuit is fully wired and ready for game logic next session.

6. Assessment

- **Live Demo Check:** Each group demonstrates their working fan-control system, explaining the sense-decide-act logic aloud.
- **Quick-Fire Questions:** Facilitator asks rotating questions during showcase — "What does the relay do here?", "What happens if you lower the threshold?"
- **Code Reading Challenge:** Show a modified snippet with a bug (e.g., wrong pin number) and ask students to spot the error.
- **Peer Review:** Groups briefly examine one another's wiring and give one improvement suggestion.
- **Exit Ticket:** One sentence — "Name one real device that uses the same sense-decide-act logic as our fan."

7. Extensions & Challenges

- **Hysteresis Challenge:** Modify code so the fan turns on above 28°C but only turns off below 25°C, preventing rapid on/off flickering.
- **Dual Threshold:** Add a second relay/LED to trigger a "warning" light at a lower temperature before the fan activates.
- **Display Upgrade:** Add an LCD or 7-segment display to show live temperature instead of only Serial Monitor.
- **Simon Says Head Start:** Early finishers may begin integrating the full game logic sketch and test single-round gameplay.

- **Data Logging:** Advanced students can log temperature readings with timestamps to design a simple "cooling report."

8. Troubleshooting Guide (FAQ)

ISSUE	LIKELY CAUSE	FIX
Sketch won't upload	Wrong board/port selected	Check Tools → Board and Tools → Port in Arduino IDE
Temperature reads "nan"	Loose DHT wiring or missing pull-up resistor	Re-seat connections; add 10kΩ pull-up if module lacks one
Relay always ON	Wired to NC instead of NO terminal, or threshold too low	Re-check terminal wiring; raise tempThreshold value
Fan won't spin at all	External battery not connected or dead	Test battery voltage with multimeter; check leads
LED very dim or not lighting	Reversed polarity or missing resistor	Flip LED orientation; confirm 220Ω resistor present
Arduino keeps resetting	Fan drawing current through Arduino 5V pin	Confirm fan is powered by separate battery, not Arduino

9. Classroom Management Tips

- Pair advanced students with those needing more support; assign rotating roles (wiring lead, code lead, tester) to keep all students engaged.
- Enforce a strict "no external power connection until facilitator checks wiring" rule — critical for relay safety.
- Use the 5-minute stretch break as a natural checkpoint to catch groups falling behind before relay wiring begins.
- Keep a visible "Help Board" where students place a red cup/card when stuck, so the facilitator can triage support efficiently.
- For early finishers, redirect immediately into the Simon Says test sketch to avoid idle time or distraction.
- Reinforce that Simon Says continues next session — frame it as "unlocking part 2," not an incomplete task, to maintain motivation.

10. Follow-Up Activities

- **Take-Home Reflection:** Students sketch one other household appliance that could use a similar sensor + relay system (e.g., automatic porch light, plant watering system).
- **Research Task:** Look up one real-world example of a relay being used in cars or home appliances and bring one fact to share next session.
- **Next Session Preview:** Full Simon Says build — completing game logic, testing multi-level gameplay, and adding personal touches (custom win/lose tunes).
- **Optional Home Challenge:** If students have spare components at home, challenge them to change the fan threshold and observe/report how sensitivity changes.